

From Domain Knowledge to Composable Reactive Code: LLM-Assisted Development with Scenario-Based Programming Supplementary Material

Adiel Ashrov¹, Tom Yaacov², and Guy Katz¹

¹ The Hebrew University of Jerusalem, Jerusalem, Israel
{adiel.ashrov,g.katz}@mail.huji.ac.il

² King's College London, London, United Kingdom
{tom.yaacov}@kcl.ac.uk

Appendix

This document provides supplementary material for the paper of the same title. Section 1 presents the formal semantics of *Scenario-Based Programming (SBP)*. Section 2 provides simplified code listings for the hand-authored SBP strategy refinements introduced in the main paper. Section 3 reports additional experimental results, referenced in the main paper, in full detail.

1 Scenario-Based Programming Semantics

The semantics of SBP are taken from [2], formally defining the concept of b-threads and their composition using labeled transition systems (LTS). Recall that an LTS is defined as a tuple $\langle S, E, \rightarrow, init \rangle$, where S is a set of states, E is a set of events, $\rightarrow \subseteq S \times E \times S$ is a transition relation, and $init \in S$ is the initial state [4]. The runs of such a transition system are sequences of the form $s^0 \xrightarrow{e^1} s^1 \xrightarrow{e^2} \dots \xrightarrow{e^i} s^i \dots$ where $s^0 = init$, and $\forall i = 1, 2, \dots, s^i \in S, e^i \in E$, and $s^{i-1} \xrightarrow{e^i} s^i$.

Definition 1 (b-thread). A *b-thread* is a tuple $\langle S, E, \rightarrow, init, R, B \rangle$ where $\langle S, E, \rightarrow, init \rangle$ forms a labeled transition system, $R: S \rightarrow 2^E$ associates a state with the set of events requested by the b-thread in that state, and $B: S \rightarrow 2^E$ associates a state with the set of events blocked in that state.

```
@bp.thread
def add_hot():
    for i in range(3):
        yield bp.sync(request=bp.BEvent("HOT"))
@bp.thread
def add_cold():
    for i in range(3):
        yield bp.sync(request=bp.BEvent("COLD"))
```

Listing 1.1. The `add_hot` and `add_cold` b-threads. Each requests to pour a small amount of hot and cold water three times. This example is adapted from Harel et al. [3].

Example 1. The `add_hot` b-thread presented in Listing 1.1 consists of four states—one for each iteration i in the loop (s_0, s_1, s_2) and a terminal state (s_3) reached when the loop ends. At each iteration, the b-thread requests the `HOT` event and blocks no event. Formally, we get that $\forall i \in \{0, 1, 2\}$: $R(s_i) = \{HOT\}, B(s_i) = \emptyset$. Once the loop ends, the b-thread reaches the terminal state, s_3 , where it does not submit statements to the event arbiter. Hence, trivially, we get that $B(s_3) = \emptyset, R(s_3) = \emptyset$.

Definition 2 (b-program). A b-program is a set of b-threads $\{\langle S_i, E_i, \rightarrow_i, init_i, R_i, B_i \rangle\}_{i=1}^n$

In line with the definitions of b-threads in [2], an abstraction is chosen in which a state of a b-thread is defined only at the synchronization points. This means that b-threads' activities between synchronization points are considered atomic and are independent of each other. We accord with Harel et al. [1], noting that this assumption is reasonable and does not unduly constrain the capabilities of b-programs.

Definition 3. (run of a b-program). A run of a b-program, $\{\langle S_i, E_i, \rightarrow_i, init_i, R_i, B_i \rangle\}_{i=1}^n$, is a run of the labeled transition system $\langle S, E, \rightarrow, init \rangle$, where $S = S_1 \times \dots \times S_n, E = \bigcup_{i=1}^n E_i, init = \langle init_1, \dots, init_n \rangle$, and $\rightarrow$ includes a transition $\langle s_1, \dots, s_n \rangle \xrightarrow{e} \langle s'_1, \dots, s'_n \rangle$ if and only if

$$e \in \bigcup_{i=1}^n R_i(s_i) \bigwedge e \notin \bigcup_{i=1}^n B_i(s_i)$$

and

$$\bigwedge_{i=1}^n ((e \in E_i \Rightarrow s_i \xrightarrow{e} s'_i) \wedge (e \notin E_i \Rightarrow s_i = s'_i)).$$

In general, multiple runs of a b-program may exist, depending on the sequence in which events are selected from the set of requested and unblocked events [2].

2 Hand-Authored SBP Strategy Refinements

Listing 1.2 shows a simplified version of the TAKI-FAMILY-PREFERENCE refinement. The b-thread assigns the same stronger selection preference to all TAKI-family cards, including both regular TAKI and Super TAKI. Other cards keep their default priority. As in the listings presented in the main paper's hand-authored SBP strategies section, the listing omits helper definitions for readability.

CONSERVE-SUPER-TAKI. The CONSERVE-SUPER-TAKI refinement combines the `prefer_taki_over_super_taki_strategy` b-thread presented in the main paper with the `block_super_taki_during_regular_taki` b-thread shown below. The first b-thread prefers regular TAKI over Super TAKI, while the second

```

@bp.thread
def taki_family_preference_strategy(index, initial_hand):
    hand = list(initial_hand)

    draw_card = player_event(index, "DrawCard", priority=20.0)
    deal_card = player_event(index, "DealCard", priority=10.0)
    no_more_cards = player_event(index, "NoMoreCards", priority=8.0)

    def strategic_play_event(card):
        priority = 5.0 if is_taki_family(card) else 10.0
        return play_event(index, card, priority=priority)

    while True:
        requested_events = [
            strategic_play_event(card)
            for card in hand
        ]

        event = yield bp.sync(
            request=requested_events,
            waitFor=[draw_card, no_more_cards]
        )

        if event.name == no_more_cards.name:
            return

        if event.name == draw_card.name:
            dealt_event = yield bp.sync(waitFor=deal_card)
            hand.append(extract_card_from_event(dealt_event))
            continue

        card = extract_card_from_event(event)
        hand.remove(card)

        if is_taki_family(card):
            yield from observe_taki_sequence_and_update(index, hand)

        if is_action_card(card):
            yield bp.sync(waitFor=BPEvent("DonePostAction"))

```

Listing 1.2. The `taki_family_preference_strategy` b-thread for one indexed player. The b-thread tracks the player's hand and re-requests TAKI and Super TAKI card-play events with priority 5.0, while other cards retain the default priority 10.0. It observes draw, deal, action completion, and game-ending events to keep its local hand state aligned with `player_behavior`.

prevents Super TAKI from being played during an open regular TAKI sequence. Together, these two b-threads define the complete refinement used in the experiments.

Listing 1.3 presents the second b-thread. Unlike the priority-based b-thread, it does not request events, but only introduces a temporary behavioral constraint. The helper `player_play_event_set(index)` matches all `Play(card)` events associated with the specified player, regardless of the card or its priority.

```
@bp.thread
def block_super_taki_during_regular_taki(index):
    play_events = player_play_event_set(index)
    closed_taki = player_event(index, "ClosedTaki")
    no_more_cards = player_event(index, "NoMoreCards")
    super_taki = play_event(index, "SuperTaki")
    while True:
        event = yield bp.sync(waitFor=[play_events, no_more_cards])
        if event.name == no_more_cards.name: return
        card = extract_card_from_event(event)
        if is_regular_taki(card):
            yield bp.sync(waitFor=closed_taki, block=super_taki)
```

Listing 1.3. The `block_super_taki_during_regular_taki` b-thread for one indexed player. The b-thread observes the player’s card-play events and, after a regular TAKI opens a sequence, blocks that player’s Super TAKI play until the sequence is closed. It terminates when the player has no remaining cards.

3 Results

This section reports additional experimental results referenced from the main paper and presented here in full detail for completeness.

3.1 SBP-Native Composition Results

The main paper reports that the generated SBP-NATIVE strategy composes with an additional hand-authored b-thread, across four self-play configurations, without observed runtime errors, deadlocks, or draws, and with no substantial imbalance in overall win rate. Table 1 reports the full position-conditioned results underlying that claim.

The self-play results remain close to parity in all four configurations. Player 1 wins between 50.4% and 50.5% of games, while Player 0 wins between 49.5% and 49.6%. No configuration produces draws, deadlocks, or runtime errors. The addition of the blocking b-thread produces no substantial change in the observed win rates in these simulations.

These results support the compositional part of RQ1 for this generated strategy: the additional b-thread composes with the LLM-generated SBP-NATIVE strategy without observed execution failures. At the same time, the results do

Table 1. Self-play comparison of SBP-NATIVE with and without the hand-authored `block_super_taki_during_regular_taki` b-thread over 10,000 games each. “Block” denotes adding the b-thread.

Configuration	Overall		Starting		Second	
	P0	P1	P0	P1	P0	P1
SBP-NATIVE vs. SBP-NATIVE	49.6%	50.4%	53.7%	54.6%	45.4%	46.3%
SBP-NATIVE vs. SBP-NATIVE+Block	49.5%	50.5%	53.9%	54.8%	45.2%	46.1%
SBP-NATIVE+Block vs. SBP-NATIVE	49.5%	50.5%	53.7%	54.8%	45.2%	46.3%
SBP-NATIVE+Block vs. SBP-NATIVE+Block	49.6%	50.4%	53.7%	54.6%	45.4%	46.3%

not show a behavioral advantage from the added b-thread. One possible explanation is that the priority structure of the generated strategy already discourages playing Super TAKI during regular TAKI sequences in the relevant states. The present experiment does not determine whether this explanation accounts for the observed result. We therefore interpret this as successful composition in simulation, but not as proof of compositional safety in general.

3.2 SBP-Native versus Hand-Authored SBP Refinements

The Strategic Performance of SBP-NATIVE subsection of the main paper additionally compares the generated SBP-NATIVE strategy against three hand-authored SBP strategy refinements: TAKI-FAMILY-PREFERENCE, TAKI-OVER-SUPER-TAKI, and CONSERVE-SUPER-TAKI. These refinements provide controlled, researcher-designed examples of how strategic knowledge can be expressed as small SBP modules. Table 2 reports the full results.

Table 2. Comparison of hand-authored SBP strategy refinements against SBP-NATIVE over 10,000 games each.

Configuration	Overall		Starting		Second	
	P0	P1	P0	P1	P0	P1
TAKI-FAMILY-PREFERENCE vs. SBP-NATIVE	47.7%	52.3%	50.2%	54.9%	45.1%	49.8%
TAKI-OVER-SUPER-TAKI vs. SBP-NATIVE	47.5%	52.5%	50.2%	55.2%	44.8%	49.8%
CONSERVE-SUPER-TAKI vs. SBP-NATIVE	47.7%	52.3%	50.7%	55.4%	44.6%	49.3%

Table 2 compares each hand-authored strategy refinement with SBP-NATIVE over 10,000 games. The three refinements produce nearly identical results, and none outperforms SBP-NATIVE, which wins between 52.3% and 52.5% of games overall. The position-conditioned results show a similar first-mover advantage for both competitors. When starting, the refinements win 50.2%–50.7% of their games, while SBP-NATIVE wins 54.9%–55.4%; when playing second, their respective win rates decrease to 44.6%–45.1% and 49.3%–49.8%. Thus, starting increases each strategy’s win rate by approximately 5.1–6.1 percentage points,

while SBP-NATIVE maintains a modest and consistent advantage regardless of starting position.

References

1. Harel, D., Lampert, R., Marron, A., Weiss, G.: Model-checking behavioral programs. In: Proceedings of the ninth ACM international conference on Embedded software. pp. 279–288. Association for Computing Machinery, New York, NY, USA (2011)
2. Harel, D., Marron, A., Weiss, G.: Programming Coordinated Behavior in Java. In: D'Hondt, T. (ed.) ECOOP 2010 – Object-Oriented Programming (2010)
3. Harel, D., Marron, A., Weiss, G.: Behavioral programming. CACM (2012)
4. Keller, R.M.: Formal verification of parallel programs. Communications of the ACM **19**(7), 371–384 (1976)